Representación eficiente de datos y estructuras dispersas

Teoría:

¿Qué es la representación eficiente de datos?

La representación eficiente de datos consiste en organizar y almacenar información de una manera que aproveche bien los recursos del computador. Los recursos más importantes en este tema son la memoria y el tiempo de procesamiento. No basta con guardar correctamente los datos: también importa cuánto espacio ocupan y qué tan rápido podemos consultarlos, recorrerlos o modificarlos.

Por ejemplo, imaginemos una tabla con 1.000 usuarios y 10.000 películas. Si cada usuario ha calificado solamente unas pocas películas, la mayoría de las combinaciones usuario-película estarán vacías. Guardar todas esas posiciones como ceros puede consumir mucha memoria sin aportar información nueva. Una estructura dispersa busca evitar ese desperdicio.

¿Qué es una matriz?

Una matriz es una organización rectangular de datos en filas y columnas. Una matriz de m filas y n columnas tiene m × n posiciones. Por ejemplo, una matriz de 3 filas y 4 columnas tiene 3 × 4 = 12 posiciones. En Python, las filas y columnas suelen numerarse desde cero.

Matrices densas y matrices dispersas

Matriz densa

Una matriz densa almacena todos los elementos de la matriz, incluidos los ceros. Es sencilla de entender y puede ser conveniente cuando una gran parte de las posiciones tiene valores útiles.

Matriz dispersa

Una matriz dispersa es una matriz en la que la mayoría de las posiciones contienen cero o no tienen un dato registrado. En vez de guardar explícitamente todos los ceros, se guardan los valores no nulos junto con información que permite conocer su ubicación.

Diferencias

Estructuras Densas: Se eligen cuando la información llena casi todo el espacio disponible. Priorizan la **velocidad de acceso directo** sobre el ahorro de memoria.

Estructuras Dispersas: Se eligen cuando el espacio está "vacío" en su mayoría. Priorizan el **ahorro drástico de memoria**, aceptando un costo adicional en el tiempo que toma buscar o modificar un elemento.

Representación mediante tripletas (COO)

Una forma sencilla de representar una matriz dispersa es guardar cada valor no nulo como una tripleta: (fila, columna, valor). COO significa Coordinate format, o formato de coordenadas.

Ventajas: es intuitivo, fácil de construir y útil para reunir entradas dispersas. Limitaciones: para algunas operaciones repetidas o acceso por filas puede resultar menos conveniente que otros formatos.

Formato CSR: almacenamiento comprimido por filas

CSR significa Compressed Sparse Row, traducido como almacenamiento comprimido por filas. Organiza los valores no nulos por filas y usa tres arreglos.

Los tres arreglos de CSR

data: contiene los valores no nulos, ordenados por filas.

índices: contiene el índice de columna correspondiente a cada valor de data.

indptr: indica dónde comienza cada fila y dónde termina dentro de data. Tiene una posición más que la cantidad de filas.

CSR es útil para recorrer filas y realizar operaciones matemáticas. No siempre es la mejor opción para insertar elementos uno por uno, porque modificar su estructura puede requerir reorganizar arreglos.

Comparación entre matriz densa, COO y CSR

Representación	Cómo guarda datos	ventaja	Limitación
Densa	Todas las posiciones, incluidos ceros	Acceso directo sencillo	Puede gastar mucha memoria si hay muchos ceros
Coo / tripletas	Fila, columna y valor por cada entrada no nula	Fácil de construir y entender	No es ideal para todas las operaciones
CRS	Valores, columnas y punteros de filas	Compacta y eficiente para recorrer filas	Inserciones individuales pueden ser costosas

Memoria: fundamento matemático

Si una matriz densa tiene m filas y n columnas, almacena m × n elementos. Si cada elemento ocupa b bytes, la memoria aproximada es:

Memoria densa ≈ m × n × b bytes

Con números de tipo float64, cada elemento ocupa normalmente 8 bytes. Una matriz de 1.000 × 1.000 elementos ocuparían aproximadamente 1.000 × 1.000 × 8 = 8.000.000 bytes, sin contar pequeños gastos adicionales de la estructura.

CSR necesita memoria para los valores, los índices de columna y los punteros de filas. Su tamaño depende del número de valores no nulos y del tipo de índices. Por eso no se debe afirmar un ahorro exacto sin medir los arreglos utilizados.

Ahorro porcentual estimado = (1 − memoria CSR / memoria densa) × 100.

Complejidad temporal y operaciones

La complejidad describe cómo crece el trabajo del algoritmo cuando aumenta el tamaño de los datos. En este tema usamos m para filas, n para columnas y nnz para el número de valores no nulos.

Recorrer una matriz densa: O(m × n), porque se visitan todas las posiciones.

Recorrer los valores almacenados en CSR: O(nnz), porque se recorren los valores no nulos.

Consultar un elemento: En una matriz densa, el acceso por índices suele ser O(1). En CSR, buscar una columna dentro de una fila puede requerir revisar sus índices; el costo depende de la fila y del método de búsqueda.

Insertar valores: COO o LIL pueden ser más cómodos durante la construcción. CSR suele ser más adecuado después de construir la matriz para operaciones y recorridos.